

Salesforce Developer Bridge Program

Day 3 Task Submission Report

Submitted By: Midasala Nikshitha Ramesh [23PA1A05F2]

Project: Hospital OPD Management System

Part 1 – Interview Warm-up

1. What is a Validation Rule?

A Validation Rule is a Salesforce feature that ensures data entered into records meets specified business requirements before it is saved. If the entered data does not satisfy the rule's condition, Salesforce prevents the record from being saved and displays an appropriate error message to the user.

2. What is a Flow?

A Flow is a declarative automation tool in Salesforce that automates business processes without requiring Apex code. It can create, update, delete, and retrieve records, send emails, perform calculations, and guide users through automated workflows based on defined conditions.

3. What is an Apex Trigger?

An Apex Trigger is custom Apex code that executes automatically before or after Salesforce records are inserted, updated, deleted, or undeleted. Triggers are used to implement complex business logic that cannot be achieved using declarative tools such as Flows or Validation Rules.

4. When would you choose a Flow instead of a Trigger?

A Flow should be chosen when the business requirement can be implemented using declarative automation without writing code. Flows are easier to develop, maintain, and modify compared to Apex Triggers. Examples include updating fields automatically, creating related records, sending email notifications, and automating approval processes.

5. Can a Validation Rule update another field? Why or why not?

No. A Validation Rule cannot update another field because its purpose is only to validate data before a record is saved. It checks whether the entered values meet predefined conditions and prevents invalid data from being saved. Updating field values requires automation tools such as Flows or Apex.

6. Which executes first: Validation Rule, Flow, or Trigger?

During record creation or update, Salesforce follows the order of execution. In general:

1. **Before-Save Record-Triggered Flow**
2. **Before Apex Trigger**
3. **Validation Rules**
4. **After Apex Trigger**
5. **After-Save Record-Triggered Flow**

This sequence ensures that data is validated before post-save automation is executed.

7. What is a Record-Triggered Flow?

A Record-Triggered Flow is a type of Salesforce Flow that automatically runs when a record is created, updated, or deleted. It is commonly used to automate business processes such as updating fields, creating related records, sending email notifications, and maintaining data consistency without writing Apex code.

Outcome

Completed the interview warm-up by reviewing fundamental Salesforce concepts, including Validation Rules, Flows, Apex Triggers, Record-Triggered Flows, and the Salesforce order of execution. This activity strengthened conceptual understanding and improved readiness for Salesforce technical interviews.

Part 2 – Business Scenario

Scenario

You are working as a Salesforce Developer. The Hospital OPD Management System requires additional automation to improve efficiency and reduce manual work. For each business requirement, the most suitable Salesforce automation tool is selected and justified.

Requirement 1: Whenever an appointment is created, an email should be sent to the Hospital Administrator.

Solution: Record-Triggered Flow

Explanation:

A Record-Triggered Flow is the most appropriate solution because it automatically sends an email whenever a new Appointment record is created. This declarative approach requires no Apex code and is easy to maintain and modify.

Requirement 2: Whenever an appointment is created, the Appointment Date should automatically be populated.

Solution: Before-Save Record-Triggered Flow

Explanation:

A Before-Save Record-Triggered Flow is the best choice because it updates the Appointment Date before the record is saved. This improves performance and avoids additional database updates.

Requirement 3: Prevent duplicate appointment records from being created.

Solution: Validation Rule

Explanation:

A Validation Rule can prevent users from saving duplicate records by checking predefined conditions before the record is saved. It ensures data quality without requiring Apex code.

Note: If duplicate checking involves complex logic across multiple related objects, an Apex Trigger may be required. For basic validation, a Validation Rule is sufficient.

Requirement 4: Validate patient and appointment details before saving.

Solution: Validation Rules

Explanation:

Validation Rules ensure that users enter valid data before records are saved. In this project, the following rules were implemented:

- Patient Age cannot be less than 0.
- Appointment Date cannot be in the past.
- Doctor Consultation Fee must be greater than 0.

These validations improve data accuracy and maintain data integrity.

Requirement 5: Whenever an appointment status changes to "Completed", automatically create a Prescription record.

Solution: After-Save Record-Triggered Flow

Explanation:

An After-Save Record-Triggered Flow is the most suitable solution because it creates a related Prescription record after the Appointment record has been successfully updated. Since a new record is being created, an After-Save Flow is required.

Outcome

Successfully analyzed the business requirements and selected the most appropriate Salesforce automation tools based on each scenario. The implemented solutions improved automation, reduced manual effort, ensured data accuracy, and enhanced the overall functionality of the Hospital OPD Management System.

Part 3 – Design Challenge

Requirement	Validation Rule	Flow	Trigger	Reason
Prevent duplicate appointment records	Yes	No	No	Validation Rules prevent duplicate or invalid data from being saved by validating user input before the record is committed to the database.
Automatically populate Appointment Date	No	Yes	No	A Before-Save Record-Triggered Flow automatically populates the Appointment Date before the record is saved, providing better performance and efficiency.
Send email notification to Hospital Administrator	No	Yes	No	A Record-Triggered Flow automatically sends an email notification whenever a new appointment is created without requiring Apex code.

Validate Patient Age, Appointment Date, and Consultation Fee	Yes	No	No	Validation Rules ensure that only valid data is saved by checking business conditions before the record is created or updated.
Automatically create a Prescription record when Appointment Status becomes Completed	No	Yes	No	An After-Save Record-Triggered Flow creates a related Prescription record automatically after the Appointment Status is updated to Completed.

Part 4 – Hands-on Assignment

Objective

To implement Record-Triggered Flows in Salesforce for automating business processes in the Hospital OPD Management System. The objective was to automatically populate the Appointment Date when a new appointment is created and send an email notification to the Hospital Administrator, thereby reducing manual effort and improving operational efficiency.

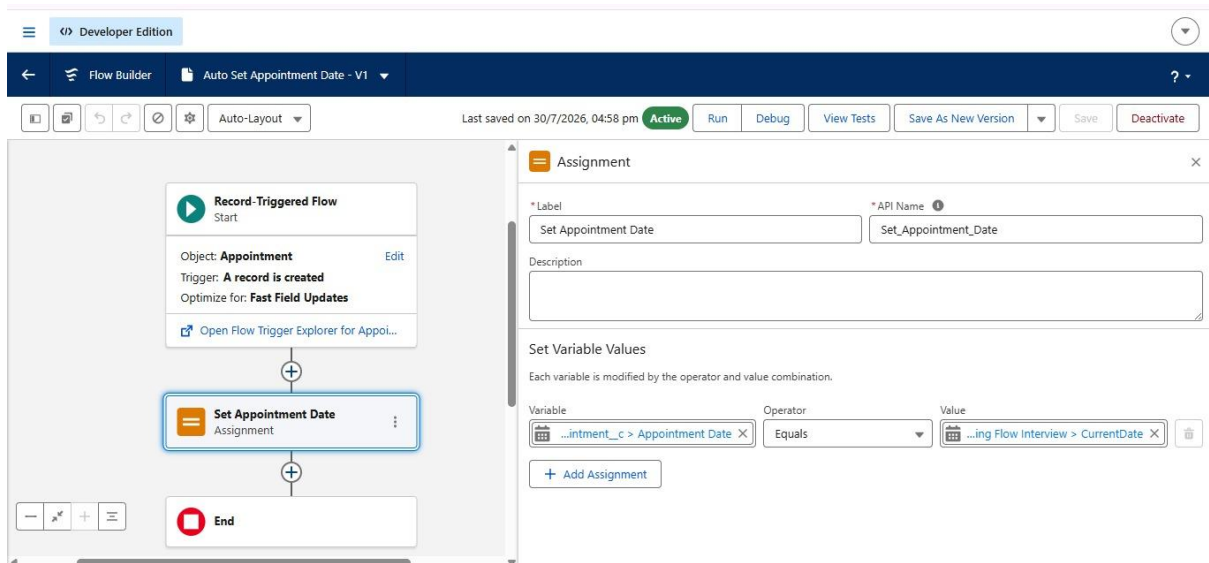
Work Performed

- Created a **Before-Save Record-Triggered Flow** on the **Appointment** object.
- Configured the flow to execute whenever a new Appointment record is created.
- Added an **Assignment** element to automatically populate the **Appointment Date** with the current date.
- Saved, activated, and tested the flow to ensure the appointment date was filled automatically during record creation.

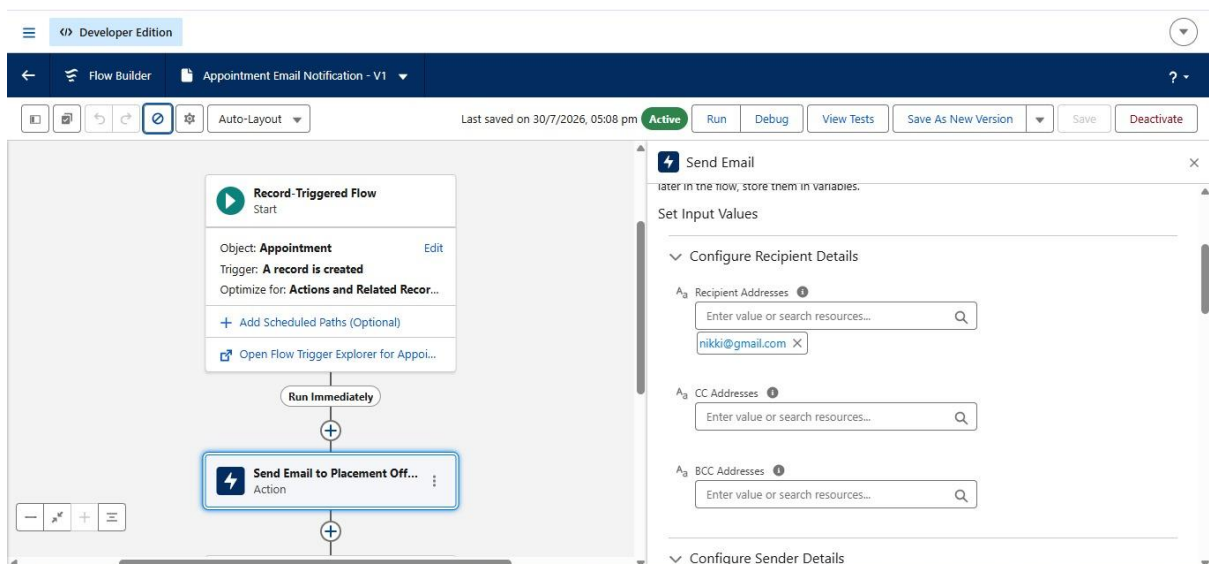
Outcome

Successfully implemented a Before-Save Record-Triggered Flow that automatically populates the Appointment Date whenever a new Appointment record is created, ensuring accurate and consistent data entry.

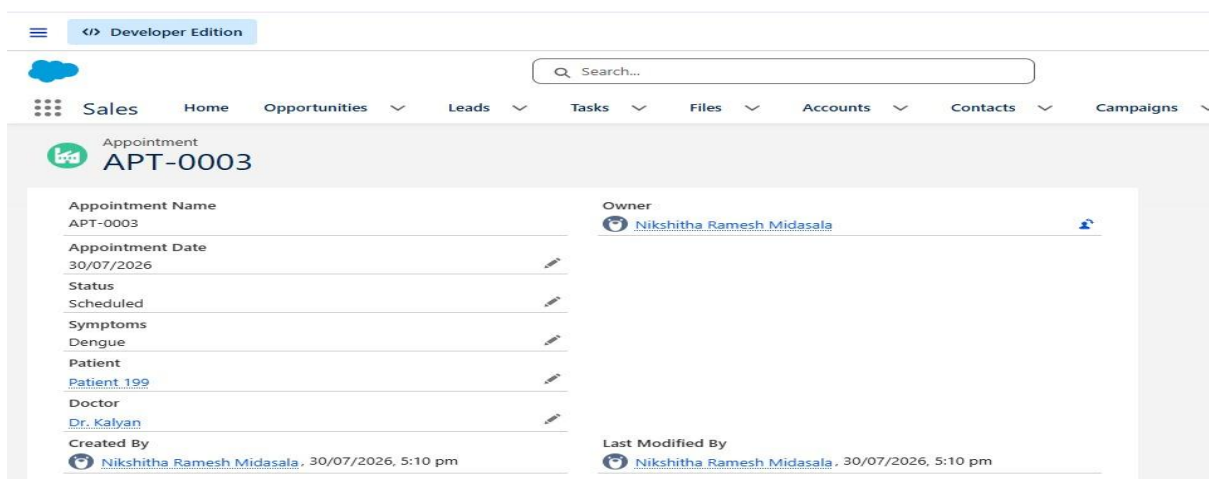
Screenshot 1.1: Auto Set Appointment Date Flow



Screenshot 1.2: Appointment Email Notification Flow



Screenshot 1.3: Successfully Created Appointment Record



Part 5 – Validation Rule Challenge

Objective

To implement Validation Rules in Salesforce to enforce business rules and ensure accurate data entry in the Hospital OPD Management System.

Work Performed

- Created a validation rule to prevent users from entering a negative value for **Patient Age**.
- Implemented a validation rule to restrict **Appointment Dates** from being set in the past.
- Created a validation rule to ensure the **Consultation Fee** is greater than zero.
- Tested each validation rule by entering invalid values and verified that Salesforce displayed the appropriate error messages.

Validation Rule Formulas

1. Patient Age Validation

Age__c < 0

2. Appointment Date Validation

Appointment_Date__c < TODAY()

3. Consultation Fee Validation

Consultation_Fee__c <= 0

Outcome

Successfully implemented and tested Validation Rules that improved data quality by preventing invalid patient age, appointment dates, and consultation fee values from being saved in Salesforce.

Screenshot 2.1: Patient Age Validation Rule

The screenshot displays the Salesforce Object Manager interface for the 'Patient Validation Rule'. The header shows 'SETUP Object Manager'. Below the header, the title 'Patient Validation Rule' is visible with a 'Back to Patient' link. The main section is titled 'Validation Rule Detail' and contains a table with the following information:

Rule Name	Age_Cannot_Be_Negative	Active	✓
Error Condition Formula	Age__c < 0		
Error Message	Age cannot be less than 0.	Error Location	Age
Description	Prevents users from entering a negative age.		
Created By	Nikshitha Ramesh Midasala, 30/07/2026, 5:17 pm	Modified By	Nikshitha Ramesh Midasala, 30/07/2026, 5:17 pm

Buttons for 'Edit' and 'Clone' are present at the top right and bottom right of the table.

Screenshot 2.2: Patient Age Validation Error

The screenshot shows a 'New Patient' form. The 'Patient Name' field contains 'Test Patient'. The 'Age' field contains '-10', which is highlighted in pink with a red 'X' icon and the message 'Age cannot be less than 0.'. The 'Gender' field is set to 'Male'. The 'Phone Number' field is empty. The 'Blood Group' field is set to 'AS+'. The 'Owner' field shows 'Nikshitha Ramesh Midasala'. A red error message box is displayed over the form, stating 'We hit a snag.' and 'Review the following fields' with a list containing 'Age'. At the bottom right, there are buttons for 'Cancel', 'Save & New', and 'Save'.

Screenshot 2.3: Appointment Date Validation Rule

The screenshot shows the 'Object Manager' interface. The 'Appointment Validation Rule' is selected. The table below shows the details of the validation rule.

Validation Rule Detail		Edit	Clone
Rule Name	Appointment_Date_Cannot_Be_Past	Active	✓
Error Condition Formula	Appointment_Date__c < TODAY()		
Error Message	Appointment Date cannot be in the past.	Error Location	Appointment Date
Description	Prevents users from selecting an appointment date in the past.		
Created By	Nikshitha Ramesh Midasala, 30/07/2026, 5:24 pm	Modified By	Nikshitha Ramesh Midasala, 30/07/2026, 5:24 pm

Screenshot 2.4: Appointment Date Validation Error

The screenshot shows the 'Edit APT-0005' form. The 'Appointment Name' field contains 'APT-0005'. The 'Appointment Date' field contains '21/07/2026', which is highlighted in pink with a red 'X' icon and the message 'Appointment Date cannot be in the past.'. The 'Status' field is set to 'Scheduled'. The 'Symptoms' field contains 'Fever'. The 'Patient' field is empty. The 'Owner' field shows 'Nikshitha Ramesh Midasala'. A red error message box is displayed over the form, stating 'We hit a snag.' and 'Review the following fields' with a list containing 'Appointment Date'. At the bottom right, there are buttons for 'Cancel', 'Save & New', and 'Save'.

Screenshot 2.5: Doctor Consultation Fee Validation Rule

The screenshot shows the Salesforce Object Manager interface for a 'Doctor Validation Rule'. The breadcrumb navigation includes 'Setup', 'Home', and 'Object Manager'. The page title is 'Doctor Validation Rule' with a 'Back to Doctor' link. Below the title is a 'Validation Rule Detail' section with 'Edit' and 'Clone' buttons. The details table shows:

Rule Name	Consultation_Fee_Positive	Active	✓
Error Condition Formula	Consultation_Fee__c <= 0		
Error Message	Consultation Fee must be greater than 0.	Error Location	Consultation Fee
Description	Prevents users from entering zero or negative consultation fees.		
Created By	Nikshitha Ramesh Midasala, 30/07/2028, 5:34 pm	Modified By	Nikshitha Ramesh Midasala, 30/07/2028, 5:34 pm

At the bottom of the details section are 'Edit' and 'Clone' buttons.

Screenshot 2.6: Consultation Fee Validation Error

The screenshot shows the 'New Doctor' form in Salesforce. The form has a header 'New Doctor' and a note '* = Required Information'. The 'Information' section contains fields for 'Doctor Name' (Dr. Faiza Tabassum), 'Specialization' (Neurologist), 'Experience' (8), and 'Consultation Fee' (₹0). The 'Consultation Fee' field is highlighted in pink, indicating a validation error. A red error message box is displayed over the field, stating: 'We hit a snag. Review the following fields: Consultation Fee'. The error message also includes the text 'Consultation Fee must be greater than 0.' Below the form, there are buttons for 'Cancel', 'Save & New', and 'Save'.

Part 6 – Trigger vs Flow Debate

Situation	Preferred Solution	Reason
Update a field automatically	Flow	Flows efficiently update field values without writing Apex code.
Create a related record	Flow	Record-Triggered Flows can automatically create

		related records after a record is saved.
Send an email notification	Flow	Flows provide built-in email actions, making implementation simple and declarative.
Call an external REST API	Trigger (Apex)	Apex is required for making HTTP callouts to external systems.
Perform complex calculations involving multiple objects	Trigger (Apex)	Apex Triggers are better suited for handling complex business logic and calculations.
Process 10,000 imported records	Trigger (Apex)	Bulkified Apex Triggers are designed to efficiently process large volumes of records while respecting Salesforce governor limits.

Part 7 – Mini Project Enhancement

Objective

To enhance the Hospital OPD Management System by automating prescription creation using a Record-Triggered Flow when an appointment is marked as **Completed**.

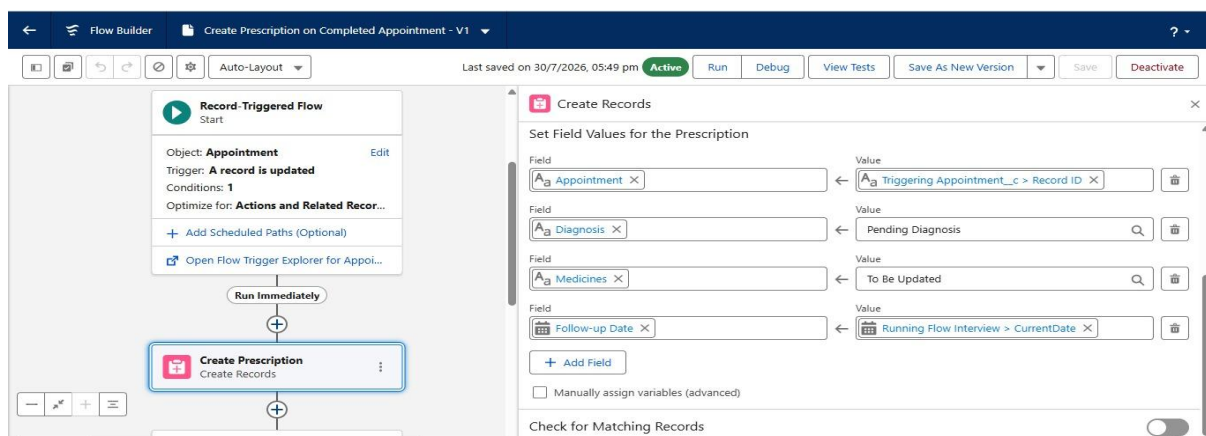
Work Performed

- Created an **After-Save Record-Triggered Flow** on the Appointment object.
- Configured the flow to automatically create a Prescription record when the Appointment Status changes to **Completed**.
- Tested the automation by updating the appointment status and verified that the Prescription record was created successfully.

Outcome

Successfully enhanced the Hospital OPD Management System by automating prescription creation, reducing manual effort and improving workflow efficiency.

Screenshot 3.1: Create Prescription Record-Triggered Flow



Screenshot 3.2: Appointment Status Updated to Completed

The screenshot shows a web application window titled "Edit APT-0005". The form contains the following fields:

- Appointment Name:** APT-0005
- Owner:** Nikshitha Ramesh Midasala (with a user icon)
- Appointment Date:** 30/07/2026 (with a calendar icon and a note "Format: 31/12/2024")
- Status:** A dropdown menu currently showing "Completed".
- Symptoms:** A text box containing "Fever".
- Patient:** A dropdown menu showing "Patient 197".
- Doctor:** A field that is currently empty.

At the bottom right of the form are three buttons: "Cancel", "Save & New", and "Save". A legend at the top right indicates "* = Required Information".

Screenshot 3.3: Automatically Created Prescription Record

The screenshot shows a CRM interface with a navigation bar at the top containing "Sales", "Home", "Opportunities", "Leads", "Tasks", "Files", "Accounts", "Contacts", and "Campaigns". The main content area displays a "Prescription" record titled "PRE-0003".

The record details are as follows:

- Prescription Name:** PRE-0003
- Owner:** Nikshitha Ramesh Midasala (with a user icon)
- Diagnosis:** Pending Diagnosis (with an edit icon)
- Medicines:** To Be Updated (with an edit icon)
- Follow-up Date:** 30/07/2026 (with an edit icon)
- Appointment:** [APT-0005](#) (with an edit icon)
- Created By:** Nikshitha Ramesh Midasala, 30/07/2026, 5:50 pm
- Last Modified By:** Nikshitha Ramesh Midasala, 30/07/2026, 5:50 pm

Part 8 – Debugging Challenge

1. What problem might occur?

If a Trigger, Flow, and Workflow Rule all update the same **Status** field, they may interfere with each other, causing unexpected record updates and making the automation difficult to manage.

2. Could automation repeatedly execute?

Yes. Updating the same field from multiple automation tools can cause repeated executions, leading to recursion, unnecessary processing, and possible governor limit issues.

3. How would you redesign this solution?

I would use a **Record-Triggered Flow** for simple automation and use an **Apex Trigger** only for complex business logic. This reduces duplicate automation, improves performance, and makes the solution easier to maintain.

Outcome

Analyzed common automation issues in Salesforce and learned how to design efficient solutions by avoiding conflicts between Flows, Triggers, and Workflow Rules.

Part 9 – Interview Questions

1. What is the difference between Workflow, Process Builder, and Flow?

- **Workflow:** Supports simple field updates, email alerts, and tasks.
 - **Process Builder:** Supports more complex automation but is now deprecated.
 - **Flow:** The most powerful declarative automation tool that can perform complex business processes and is the recommended solution in Salesforce.
-

2. Why is Flow replacing Workflow Rules?

Flow provides more advanced features, greater flexibility, and can handle complex business logic that Workflow Rules cannot.

3. What is a Record-Triggered Flow?

A Record-Triggered Flow is an automation that runs automatically when a record is created, updated, or deleted.

4. What are Before-Save and After-Save Flows?

- **Before-Save Flow:** Used to update fields before the record is saved.
 - **After-Save Flow:** Used to create related records, send emails, or perform actions after the record is saved.
-

5. When should Apex be preferred over Flow?

Apex should be used when the business logic is complex, requires external API callouts, or cannot be implemented using declarative tools.

6. Can Flow call Apex?

Yes. A Flow can invoke Apex Actions to execute custom business logic.

7. What are the advantages of declarative automation?

Declarative automation reduces coding, is easier to maintain, improves productivity, and follows Salesforce best practices.

8. Explain one Flow that you built.

I built a Record-Triggered Flow that automatically creates a Prescription record when an Appointment Status changes to **Completed**.

9. Explain one Validation Rule that you created.

I created a Validation Rule to prevent users from entering a negative Patient Age, ensuring only valid data is saved.

10. If given the choice, why did you use Flow instead of Apex?

I used Flow because it met the business requirements without coding, making the solution easier to develop, maintain, and modify.

Outcome

Reviewed and answered common Salesforce interview questions to strengthen conceptual knowledge and improve readiness for Salesforce Developer interviews.

Conclusion

Successfully completed all Day 3 activities of the Salesforce Interview Readiness Bootcamp, including conceptual discussions, Record-Triggered Flow implementation, Validation Rules, automation design, debugging scenarios, and interview preparation. The hands-on exercises strengthened my understanding of Salesforce declarative automation, improved data quality through validation, and enhanced the Hospital OPD Management System by automating key business processes. Overall, this assignment improved both my practical Salesforce development skills and my confidence in applying automation solutions to real-world business requirements.

GitHub Repo Link:

<https://github.com/Nikkiramesh424/salesforce-self-learning>